

Building a Chatbot UI with Streamlit

Splitting a LangGraph chatbot into a reusable backend and a Streamlit frontend, and solving the classic conversation-history-reset bug with session state.

WHAT THIS CHAPTER COVERS

- ▶ Backend / frontend separation for a LangGraph chatbot
- ▶ `st.chat_message` and `st.chat_input` components
- ▶ Streamlit's rerun-on-interaction execution model
- ▶ The message history reset bug and its root cause
- ▶ `st.session_state` as persistent per-session storage
- ▶ Wiring session state to the message history list
- ▶ Importing and invoking the LangGraph chatbot object
- ▶ Passing `thread_id` via config for checkpointer continuity

Building a Chatbot UI with Streamlit

RECAP — CHAPTER 10

Chapter 10 covered **persistence** in LangGraph: checkpoints, threads, fault tolerance, and time travel, using an `InMemorySaver` and a `thread_id`-scoped config to make graph state durable across invocations. This chapter reuses that exact persistence pattern — the chatbot built earlier already has short-term memory via a checkpointer, but it has no user interface. This chapter gives it one.

1. The Missing Piece: A User Interface

The Problem With the Notebook-Only Chatbot

The LangGraph chatbot built in the previous chapters worked correctly and retained short-term memory across turns using a checkpointer. However, it had one major limitation: it had **no user interface**. Every interaction happened inside the same Jupyter notebook where the graph was defined, which is not how a real user would ever use a chatbot.

This chapter fixes that gap by building a proper *web interface* using **Streamlit** — a Python library for building websites and interactive UIs without needing separate frontend languages like HTML, CSS, or JavaScript. The result is a chat website where a user can type messages, see the full conversation rendered as chat bubbles, and get real LangGraph-powered responses with working short-term memory.

Splitting the Chatbot Into Two Components

To build this UI, the existing single-file chatbot must be split into **two components**:

1. **Backend** (`langgraph_backend.py`) — contains the LangGraph workflow: the state definition, the chat node, the graph construction, and the compiled `chatbot` object with its checkpointer. This is almost entirely code that was already written in earlier chapters.
2. **Frontend** (`streamlit_frontend.py`) — contains the Streamlit UI code: displaying chat bubbles, capturing user input, and calling the backend's `chatbot.invoke()` to get responses.

The user interacts only with the frontend by typing a message. The frontend forwards that message to the LangGraph backend, receives the response, and renders it back to the user. The bulk of the new engineering work in this chapter is on the frontend side, since the backend is mostly a direct reuse of the chatbot built earlier.

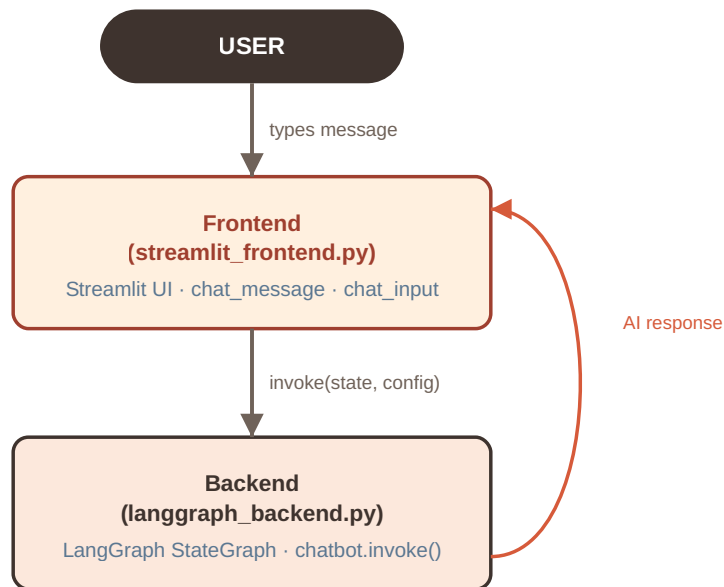


Figure 11.1 — Two-tier architecture — the frontend forwards user input to the backend's `chatbot.invoke()` and renders the returned AI message.

Prerequisites

This chapter assumes basic familiarity with Streamlit fundamentals (covered briefly here, but a short external tutorial helps) and a working understanding of LangGraph from the earlier chapters in this series.

Quick Revision — The Missing Piece

- The earlier chatbot worked correctly but had no user interface; interaction happened inside a notebook.
- Streamlit is a Python library for building interactive web UIs without HTML/CSS/JS.
- The chatbot is split into two files: `langgraph_backend.py` (LangGraph workflow) and `streamlit_frontend.py` (UI).
- The frontend calls `chatbot.invoke()` from the backend and displays the returned response.

2. Core Streamlit Chat Components

Before wiring up LangGraph, it is necessary to understand the two Streamlit components that make up any chat-style interface.

1. `ST.CHAT_MESSAGE` — DISPLAYING A MESSAGE BUBBLE

Every message shown on screen — from the user or from the assistant — is displayed using `st.chat_message(role)`, where `role` is either `"user"` or `"assistant"`. Streamlit automatically renders a distinct icon per role so it is immediately clear who sent which message.

```
import streamlit as st

# Display a single user message bubble
with st.chat_message("user"):
    st.text("Hi")

# Display a single assistant message bubble
with st.chat_message("assistant"):
    st.text("How can I help you?")
```

2. ST.CHAT_INPUT — CAPTURING USER INPUT

The bottom input box where the user types their message is created with `st.chat_input(placeholder)`. It returns the typed text once the user presses Enter, or `None` if nothing has been submitted yet.

```
user_input = st.chat_input("Type here")

if user_input:
    with st.chat_message("user"):
        st.text(user_input)
```

st.chat_message(role)

Displays one message bubble with role icon

st.chat_input(placeholder)

Bottom input box; returns typed text

Figure 11.2 — The two building blocks of any Streamlit chat UI: message bubbles and the input box.

IMPORTANT

These two components alone are not enough to build a working multi-turn chatbot. `st.chat_message` only renders whatever it is given at the moment it runs — it does not remember anything across interactions on its own. The next section explains why that matters.

🔄 Quick Revision — Core Chat Components

- `st.chat_message(role)` renders one message bubble; role is 'user' or 'assistant', each with its own icon.
- `st.chat_input(placeholder)` renders the bottom input box and returns the typed text on Enter.
- `st.chat_input` returns `None` until the user actually submits a message.
- These components render whatever they are given at that instant; they carry no memory of their own.

3. Streamlit's Rerun Model and the History-Reset Bug

A Naive First Attempt: The "Copycat" Chatbot

A natural first attempt at a multi-turn chatbot is to keep a plain Python list of messages, append to it, and loop over it to display the whole conversation:

```
message_history = []

user_input = st.chat_input("Type here")

if user_input:
    message_history.append({"role": "user", "content": user_input})
    with st.chat_message("user"):
        st.text(user_input)

    # For now: assistant just echoes the same input back
    message_history.append({"role": "assistant", "content": user_input})
    with st.chat_message("assistant"):
        st.text(user_input)

for message in message_history:
    with st.chat_message(message["role"]):
        st.text(message["content"])
```

This "copycat" version deliberately echoes the user's own input back as the assistant's reply, purely to validate the display logic before connecting to a real LLM. Each message is stored as a dictionary with a `role` key and a `content` key, and all such dictionaries are collected in a Python list, `message_history`.

Why the History Disappears on the Second Message

Running this code reveals a bug: after typing a second message, the first message disappears entirely, leaving only the newest user/assistant pair on screen.

COMMON MISTAKE

The cause is Streamlit's execution model: **every time the user submits input (presses Enter), the entire Python script reruns from top to bottom.** The line `message_history = []` sits at the top of the script, so on every rerun it silently resets the list back to empty before the append logic runs. The conversation history was never actually being stored anywhere persistent — it was being recreated and destroyed on every single interaction.

🔄 Quick Revision — Rerun Model & the Bug

- Streamlit reruns the entire script top-to-bottom on every user interaction (e.g. pressing Enter in `chat_input`).
- A plain Python list defined at the top of the script gets reinitialized to empty on every rerun.
- This causes previously displayed messages to disappear as soon as a new message is submitted.
- The root cause is that conversation history was never stored anywhere that survives a rerun.

4. Fixing It With `st.session_state`

Definition

`st.session_state` is a Streamlit component that behaves like a dictionary, but with one crucial difference from an ordinary Python dictionary: **its contents are not reset when the script reruns**. Values stored in `st.session_state` persist for the lifetime of the user's browser session, and are only cleared when the page is manually refreshed.

How It Works

The fix follows a simple pattern: check whether a key already exists in `st.session_state`; if not, initialize it once. Because this initialization only happens the first time (subsequent reruns find the key already present and skip re-initializing it), the stored list survives every future rerun.

```
# st.session_state behaves like a dict, but persists across reruns
if 'message_history' not in st.session_state:
    st.session_state['message_history'] = []
```

All subsequent reads and writes then go through `st.session_state['message_history']` instead of a plain local list:

```
# Display the full history first, on every rerun
for message in st.session_state['message_history']:
    with st.chat_message(message["role"]):
        st.text(message["content"])

user_input = st.chat_input("Type here")

if user_input:
    # Add to session_state history first, then display
    st.session_state['message_history'].append({'role': 'user', 'content': user_input})
    with st.chat_message('user'):
        st.text(user_input)

    st.session_state['message_history'].append({'role': 'assistant', 'content': user_input})
    with st.chat_message('assistant'):
        st.text(user_input)
```

With this change, every rerun first replays the entire stored history from `st.session_state` (so nothing is lost), then appends and displays the new user and assistant turns on top of it.

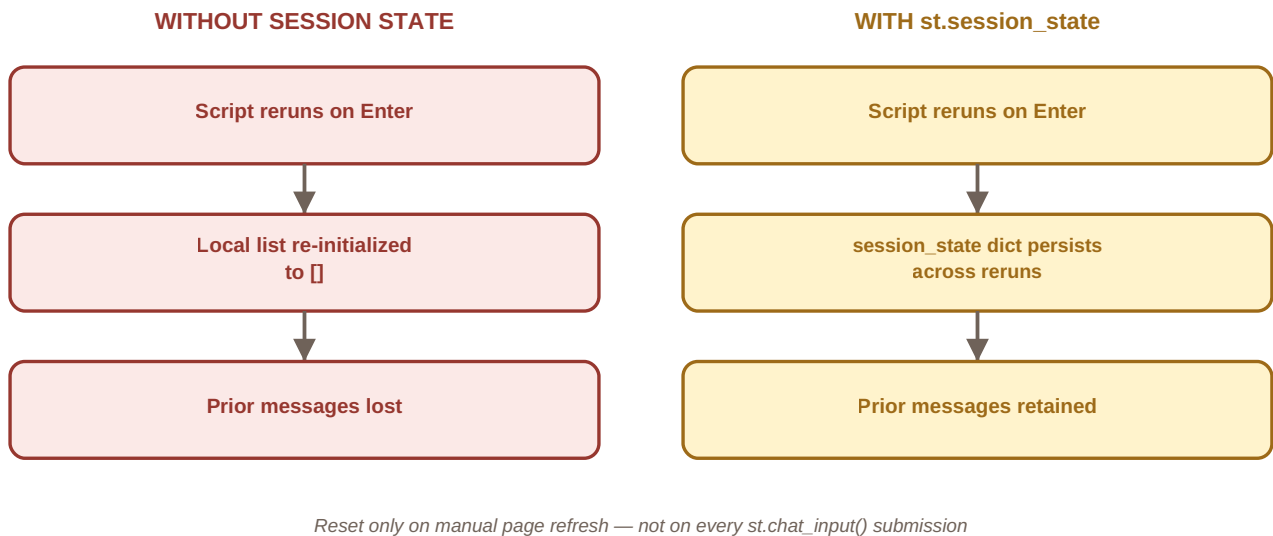


Figure 11.3 — A plain list resets on every Streamlit rerun; `st.session_state` persists across reruns.

BEST PRACTICE

Always add a new message to `st.session_state['message_history']` *before* displaying it in the current run. This keeps the "replay full history, then show the new turn" pattern consistent and avoids duplicated or missing messages.

🔄 Quick Revision — `st.session_state`

- `st.session_state` is a dict-like Streamlit object whose contents survive script reruns.
- It resets only on a manual browser page refresh, not on every `chat_input` submission.
- Pattern: initialize a key only if 'key' not in `st.session_state`, so it isn't wiped every rerun.
- On every rerun, the full stored history is replayed first, then the new turn is appended and shown.

5. Connecting Streamlit to the LangGraph Backend

The Backend, As-Is

The backend file is almost exactly the chatbot built in earlier chapters: a `MessageState` TypedDict with an `add_messages-reduced` list, a single `chat_node`, a two-edge graph (`START` → `chat_node` → `END`), and a compiled chatbot object using an `InMemorySaver` checkpointer.

```

from langchain_core.messages import HumanMessage, BaseMessage
from langgraph.graph import StateGraph, START, END
from langgraph.graph.message import add_messages
from langgraph.checkpoint.memory import InMemorySaver
from langchain_groq import ChatGroq
from typing import TypedDict, Annotated
from dotenv import load_dotenv

load_dotenv()
model = ChatGroq(model = "openai/gpt-oss-20b")

class MessageState(TypedDict):
    messages : Annotated[list[BaseMessage], add_messages]

def chat_node(state : MessageState):
    messages = state['messages']
    response = model.invoke(messages).content
    return {'messages':[response]}

graph = StateGraph(MessageState)
graph.add_node('chat_node', chat_node)
graph.add_edge(START, 'chat_node')
graph.add_edge('chat_node', END)

checkpointer = InMemorySaver()
chatbot = graph.compile(checkpointer = checkpointer)

```

Nothing in this backend needs to change to support the UI — it is reused directly by importing the compiled chatbot object.

Importing the Backend Into the Frontend

The frontend imports the compiled graph object directly by module and variable name:

```

from langgraph_backend import chatbot
from langchain_core.messages import HumanMessage

```

Replacing the Echo Logic With a Real Invoke Call

Of the whole frontend, only **one place** needs to change to go from the "copycat" dummy chatbot to a real AI-powered one: the block that produces and displays the assistant's message. Everywhere else — loading history, displaying the user's message, appending to `session_state` — stays exactly as already written.

The user's typed text must be wrapped in a `HumanMessage` and passed to `chatbot.invoke()` as the initial state, under the `messages` key (matching the backend's `MessageState` schema):

```

response = chatbot.invoke({'messages': [HumanMessage(content=user_input)]})
ai_message = response['messages'][-1].content

```


`chatbot.invoke()` returns the full updated state; the newest message in that state's `messages` list is the assistant's reply, so it is extracted with `[-1].content`.

COMMON MISTAKE

Calling `chatbot.invoke()` with only the `messages` state and no `config` raises an error, because the graph was compiled with a checkpointer. A checkpointer requires a `thread_id` at invoke time to know which conversation's checkpoint to read and write. Forgetting the config is a common oversight when wiring a checkpointed graph into a new frontend.

Passing the Thread Config

The fix is to define a config dictionary with a `thread_id` and pass it into every `invoke()` call:

```
CONFIG = {'configurable': {'thread_id': 'thread_1'}}

response = chatbot.invoke(
    {'messages': [HumanMessage(content=user_input)]},
    config = CONFIG
)
ai_message = response['messages'][-1].content
```

With a fixed `thread_id`, the checkpointer consistently reads and appends to the same conversation thread across every rerun, which is what gives the deployed chatbot its short-term memory — it can correctly answer follow-up questions like "What is my name?" after the user introduced themselves earlier in the same session.

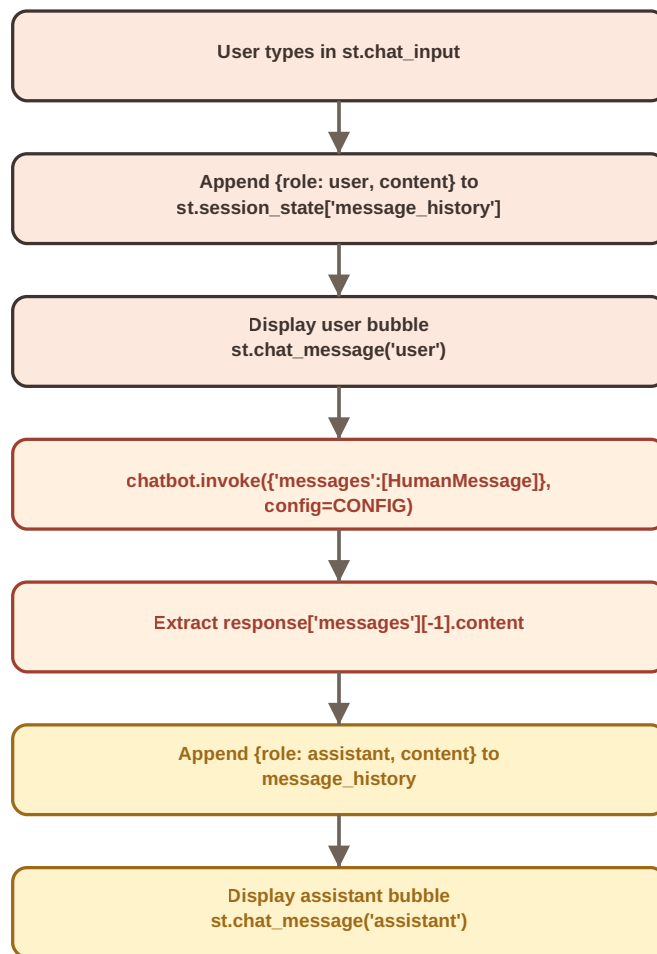


Figure 11.4 — The complete round trip for one user turn, from input to persisted assistant reply.

INTERVIEW TIP

If asked why a Streamlit + LangGraph chatbot "forgets" earlier turns despite using a checkpointer, check two things: (1) is `st.session_state` being used to persist the UI-level display history, and (2) is a consistent `thread_id` being passed in the `config` on every `invoke()` call. Both are required — `session_state` handles what's shown on screen, while the checkpointer + `thread_id` handles what the LLM actually receives as conversation context.

🔁 Quick Revision — Connecting to LangGraph

- The backend (`langgraph_backend.py`) is reused as-is; only its compiled chatbot object is imported.
- Only the assistant-message block in the frontend needs to change to use a real LLM instead of an echo.
- `chatbot.invoke({'messages':[HumanMessage(content=...)]}, config=CONFIG)` returns the full updated state.
- The AI reply is `response['messages'][-1].content`.
- A checkpointed graph requires a `config` with a `thread_id` at every `invoke()` call, or it raises an error.
- A fixed `thread_id` keeps the whole UI session mapped to one continuous LangGraph conversation thread.

6. Full Frontend Code

Putting every piece from this chapter together produces the complete `streamlit_frontend.py`:

```
import streamlit as st
from langgraph_backend import chatbot
from langchain_core.messages import HumanMessage

# Define config
CONFIG = {'configurable':{'thread_id':'thread_1'}}

# st.session_state -> dict
if 'message_history' not in st.session_state:
    # Define message history in the format = [{'role':'...', 'content':'..'}]
    st.session_state['message_history'] = []

# Showing the whole message history before showing the current messages
for message in st.session_state['message_history']:
    with st.chat_message(message['role']):
        st.text(message['content'])

# Input box
user_input = st.chat_input('Type here')

if user_input:
    # Add the message to message history first
    st.session_state['message_history'].append({'role':'user', 'content':user_input})
    with st.chat_message('user'):
        st.text(user_input)

    # The question first goes to LLM
    response = chatbot.invoke({'messages':[HumanMessage(user_input)]}, config = CONFIG)
    ai_message = response['messages'][-1].content

    # Add the message to message history first
    st.session_state['message_history'].append({'role':'assistant', 'content':ai_message})
    with st.chat_message('assistant'):
        st.text(ai_message)
```

Running the App

Unlike a notebook cell, a Streamlit app is launched from the terminal, which starts a local web server and opens the UI in the browser:

```
streamlit run streamlit_frontend.py
```

The `.env` file holding the model provider's API key (e.g. `GROQ_API_KEY`) must sit alongside the backend/frontend files so `load_dotenv()` in the backend can pick it up. Required libraries for this setup are `langchain`, `langgraph`, and `streamlit`, installed in a virtual environment.

IMPORTANT

Never hardcode a real API key inside the source files. Keep it only in the `.env` file (excluded from version control) and load it via `load_dotenv()` — the actual key value should never appear as a literal string in `langgraph_backend.py` or anywhere else in the codebase.

🔄 Quick Revision — Full Frontend

- The complete frontend combines: `session_state` init, history replay loop, `chat_input`, `invoke` with `CONFIG`, and history append for both roles.
- A Streamlit app is launched from the terminal with: `streamlit run .py`
- The `.env` file with the LLM provider's API key must be accessible via `load_dotenv()` in the backend.
- Required packages: `langchain`, `langgraph`, `streamlit` (plus the specific model-provider integration package).

K. Key Takeaways

- **Backend/Frontend Split** — A LangGraph chatbot is separated into a backend (the graph, state, and compiled chatbot object) and a frontend (the UI), letting the frontend simply import and reuse the backend's compiled object.
- **`st.chat_message`** — Renders one message bubble for a given role ("user" or "assistant"), each with a distinct icon, but has no memory of its own.
- **`st.chat_input`** — Renders the bottom text box and returns the typed text once submitted, or `None` otherwise.
- **Streamlit's Rerun Model** — The entire script reruns top-to-bottom on every user interaction, which silently resets any plain local variable defined at the top of the script.
- **The History-Reset Bug** — A plain Python list for message history gets wiped on every rerun, causing older messages to disappear as soon as a new one is submitted.
- **`st.session_state`** — A dict-like Streamlit object that persists across reruns and resets only on a manual page refresh, making it the correct place to store conversation history.
- **Connecting to LangGraph** — Only the assistant-response block needs a real `chatbot.invoke()` call; the user's text is wrapped in `HumanMessage` and sent under the `messages` key.
- **Thread Config** — A checkpointed graph requires a config with a fixed `thread_id` on every `invoke()` call, which is what gives the UI session continuous short-term memory.

R. Revision Sheet

Backend / Frontend Split

Backend holds the LangGraph workflow and compiled chatbot object; frontend holds the Streamlit UI and imports the backend directly.

st.chat_message(role)

Displays one chat bubble for "user" or "assistant" with a role-specific icon; must be given content on every rerun to reappear.

st.chat_input(placeholder)

Renders the bottom input box; returns the submitted text (or None) and triggers a script rerun on Enter.

Streamlit Rerun Model

The whole script re-executes top-to-bottom on every user interaction — the source of the history-reset bug.

st.session_state

Dict-like store that survives reruns; initialize a key only if it is not already present so it is not overwritten each run.

chatbot.invoke()

Called with {'messages':[HumanMessage(...)]} and a config carrying thread_id; returns the full updated state.

Extracting the Reply

response['messages'][-1].content gives the newest (assistant) message from the invoke() result.

thread_id / CONFIG

{'configurable':{'thread_id':'thread_1'}} — required by any graph compiled with a checkpoint; keeps one continuous conversation.

I. Interview Questions

Q1. Why is a LangGraph chatbot typically split into a separate backend and frontend file when adding a UI?

Q2. What is the difference between st.chat_message and st.chat_input in Streamlit?

Q3. Explain Streamlit's rerun execution model and why it matters for stateful UIs.

Q4. Why does a plain Python list used for chat history get reset every time a new message is submitted?

Q5. What is st.session_state, and how does its lifecycle differ from an ordinary local variable in a Streamlit script?

Q6. What condition should be checked before initializing a key in st.session_state, and why?

Q7. Why must `chatbot.invoke()` include a config with a `thread_id` when the graph was compiled with a checkpointer?

Q8. How is the assistant's reply extracted from the object returned by `chatbot.invoke()`?

Q9. What would happen if the `thread_id` in `CONFIG` were changed to a new value mid-conversation?

Q10. Why should API keys be stored in a `.env` file rather than hardcoded into the backend script?

F. Further Reading

- [Streamlit official documentation — Chat elements \(`st.chat\_message`, `st.chat\_input`\)](#)
- [Streamlit official documentation — Session State API reference](#)
- [LangGraph official documentation — Persistence and checkpointers](#)
- [LangChain official documentation — HumanMessage and message types](#)